# Modelo de objetos predefinidos en JavaScript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

**Antonio** ha completado correctamente la fase de introducción y fundamentos básicos del lenguaje JavaScript, y ahora comienza a investigar en las características de los objetos predefinidos en JavaScript.

Estos objetos le van a permitir gestionar ventanas, marcos, propiedades de los navegadores, de las URL, etc. en JavaScript.

Además, también va a poder realizar operaciones matemáticas, de fecha y de cadenas, con otros tantos objetos nativos del lenguaje JavaScript.

**Antonio** tiene una pequeña reunión con **Ada** y con su tutor **Juan**, para comentar los progresos realizados hasta este momento y se pone manos a la obra con esta nueva sección.

En esta unidad de trabajo profundizaremos en los objetos de más alto nivel en JavaScript, los cuales estarán disponibles en todas nuestras aplicaciones. Veremos como gestionar los marcos o frames y la forma de comunicarnos entre múltiples ventanas que tengamos abiertas en el navegador.

Para terminar con la unidad veremos los objetos nativos en JavaScript, aquellos que están definidos en el propio lenguaje y que nos van a permitir realizar operaciones matemáticas complejas, trabajar con cadenas de texto, operaciones de fecha y hora, etc.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Ministerio de Educación, Formación Profesional y Deportes (Dominio público)

**Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.**

Aviso Legal

# 1.- Objetos de más alto nivel en Javascript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

Bajo la tutoría de **Juan**, **Antonio** se dispone a profundizar en los objetos básicos y de más alto nivel de JavaScript.

Estos objetos, los encontrará en prácticamente la mayoría de aplicaciones que haga con JavaScript, por lo que será fundamental, que tenga muy claras las características y diferentes funcionalidades que estos objetos le van a aportar a sus aplicaciones.

Una página web, es un documento HTML que será interpretado por los navegadores de forma gráfica, pero que también va a permitir el acceso al código fuente de la misma.

El Modelo de Objetos del Documento (DOM), permite ver el mismo documento de otra manera, describiendo el contenido del documento como un conjunto de objetos, sobre los que un programa de Javascript puede interactuar.

Según el W3C, el Modelo de Objetos del Documento es una interfaz de programación de aplicaciones (API), para documentos válidos HTML y bien construidos XML. Define la estructura lógica de los documentos, y el modo en el que se acceden y se manipulan.

Ahora que ya has visto en la unidad anterior, los fundamentos de la programación, vamos a profundizar un poco más en lo que se refiere a los objetos, que podremos colocar en la mayoría de nuestros documentos.

Definimos como **objeto**, una entidad con una serie de **propiedades** que definen su estado, y unos **métodos** (funciones), que actúan sobre esas propiedades.

La forma de acceder a una propiedad de un objeto es la siguiente:

```
nombreObjeto.propiedad
```

La forma de acceder a un método de un objeto es la siguiente:

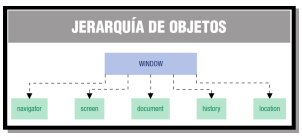
```
nombreObjeto.metodo( [parámetros opcionales] )
```

También podemos referenciar a una propiedad de un objeto, por su índice en la creación. Los índices comienzan por 0.

En esta unidad, nos enfocaremos en objetos de alto nivel, que encontrarás frecuentemente en tus aplicaciones de JavaScript: **window**, **location**, **navigator** y **document**. El objetivo, no es solamente indicarte las nociones básicas para que puedas comenzar a realizar tareas sencillas, sino también, el prepararte para profundizar en las propiedades y métodos, gestores de eventos, etc. que encontrarás en unidades posteriores.

En esta unidad, verás solamente las propiedades básicas, y los métodos de los objetos mencionados anteriormente.

Te mostramos aquí el gráfico del modelo de objetos de alto nivel, para todos los navegadores que permitan usar JavaScript.




Ministerio de Educación (CC BY-NC-SA)

Es muy importante que tengas este gráfico en mente porque va a ser la guía a lo largo de toda esta unidad.

En esta unidad no hablaremos del uso de **frames** ya que su uso está totalmente desaconsejado por motivos de accesibilidad y usabilidad, por lo que te recomendamos no usarlos.

## Autoevaluación

**El nombre de un método en JavaScript siempre lleva paréntesis al final:**

- ○ Sí.
- ○ No.
- ○ Depende de si lleva o no parámetros.

Es correcta. Todos los métodos de cualquier objeto en JavaScript se nombrarán con el nombre del método seguido de paréntesis y sus parámetros opcionales.

Es incorrecta. Si no lleva paréntesis se consideraría una propiedad.

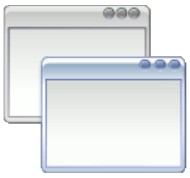
No es correcta. Aunque no lleve parámetros tendría que llevar los paréntesis igualmente.

# Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

TODO: Falta ejemplo de consulta de los objetos desde la consola del navegador. Para saber más de iframes.

# 1.1.- Objeto window



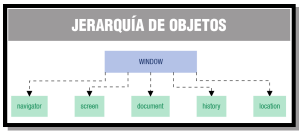
Everaldo Coelho (GNU/GPL)

En la jerarquía de objetos, tenemos en la parte superior el objeto **window**.

Este objeto está situado justamente ahí, porque es el contenedor principal de todo el contenido que se visualiza en el navegador. Tan pronto como se abre una ventana (**window**) en el navegador, incluso aunque no se cargue ningún documento en ella, este objeto window ya estará definido en memoria.

Además de la sección de contenido del objeto **window**, que es justamente dónde se cargarán los documentos, el campo de influencia de este objeto, abarca también las dimensiones de la ventana, así como todo lo que rodea al área de contenido: las barras de desplazamiento, barra de herramientas, barra de estado, etc.

Cómo se ve en el gráfico de la jerarquía de objetos, debajo del objeto **window** tenemos otros objetos como el **navigator**, **screen**, **history**, **location** y el objeto **document**. Este objeto **document** será el que contendrá toda la jerarquía de objetos, que tengamos dentro de nuestra página HTML.




Ministerio de Educación (CC BY-NC-SA)

**Atención** En los navegadores más modernos, los usuarios tienen la posibilidad de abrir las páginas tanto en nuevas pestañas dentro de un navegador, como en nuevas ventanas de navegador. Para JavaScript tanto las ventanas de navegador, como las pestañas, son ambos objetos **window**.

**Acceso a propiedades y métodos.**

Para acceder a las propiedades y métodos del objeto **window**, lo podremos hacer de diferentes formas, dependiendo más de nuestro estilo, que de requerimientos sintácticos. Así, la forma más lógica y común de realizar esa referencia, incluiría el objeto **window** tal y como se muestra en este ejemplo:

```
window.nombrePropiedad
window.nombreMétodo( [parámetros] )
```

Como puedes ver, los parámetros van entre corchetes, indicando que son opcionales y que dependerán del método al que estemos llamando.

Un objeto **window** también se podrá referenciar mediante la palabra **self**, cuando estamos haciendo la referencia desde el propio documento contenido en esa ventana:

```
self.nombrePropiedad
self.nombreMétodo( [parámetros] )
```

Podremos usar cualquiera de las dos referencias anteriores, pero intentaremos dejar la palabra reservada **self**, para scripts más complejos en los que tengamos múltiples marcos y ventanas.

Debido a que el objeto **window** siempre estará presente cuando ejecutemos nuestros scripts, podremos omitirlo, en referencias a los objetos dentro de esa ventana. Así que, si escribimos:

```
nombrePropiedad
nombreMétodo( [parámetros] )
```

También funcionaría sin ningún problema, porque se asume que esas propiedades o métodos, son del objeto de mayor jerarquía (el objeto **window**) en el cuál nos encontramos.

## Citas para pensar

“Sólo cerrando las puertas detrás de uno se abren ventanas hacia el porvenir.” ***SAGAN, Françoise.***

## Para saber más

Otro objeto de alto nivel interesante es `screen`. Screen representa la pantalla del equipo y permite obtener información sobre esta. Puedes profundizar sobre este objeto en:

- Documentación del objeto Screen.
- Ejemplos interactivos de uso del objeto Screen (w3school).

# 1.1.1.- Gestión de ventanas



Everaldo Coelho (GNU/GPL)

Un script no creará nunca la ventana principal de un navegador. Es el usuario, quien realiza esa tarea abriendo una URL en el navegador o un archivo desde el menú de abrir. Pero sin embargo, un script que esté ejecutándose en una de las ventanas principales del navegador, si que podrá crear o abrir nuevas sub-ventanas.

El método que genera una nueva ventana es **window.open()**. Este método contiene hasta tres parámetros, que definen las características de la nueva ventana: la URL del documento a abrir, el nombre de esa ventana y su apariencia física (tamaño, color,etc.).

Por ejemplo, si consideramos la siguiente instrucción que abre una nueva ventana de un tamaño determinado y con el contenido de un documento HTML:

```
let subVentana = window.open("nueva.html", "nueva", "height=800, width=600");
```

Lo importante de esa instrucción, es la asignación que hemos hecho en la variable **subVentana**. De esta forma podremos a lo largo de nuestro código, referenciar a la nueva ventana desde el script original de la ventana principal. Por ejemplo, si quisiéramos cerrar la nueva ventana desde nuestro script, simplemente tendríamos que hacer: **subVentana.close();**

Aquí si que es necesario especificar **subVentana**, ya que si escribiéramos **window.close()**, **self.close()** o **close()** estaríamos intentando cerrar nuestra propia ventana (previa confirmación), pero no la **subVentana** que creamos en los pasos anteriores.

**Véase el siguiente ejemplo que permite abrir y cerrar una sub-ventana:**

```
<!DOCTYPE  html>
<html>
    <head>
        <meta  http-equiv="content-type"  content="text/html;charset=utf-8">
        <title>Apertura y Cierre de Ventanas</title>
        <script type="text/javascript">
            const inicializar = () => {
                document.getElementById("crear-ventana").onclick=crearNueva;
                document.getElementById("cerrar-ventana").onclick=cerrarNueva;
            }

            let  nuevaVentana;

            const crearNueva = () =>  {
                nuevaVentana  =  window.open("http://www.google.es","","height=400,width=800");
            }

            const cerrarNueva = () =>  {
                if  (nuevaVentana) {
                    nuevaVentana.close(); nuevaVentana  =  null;
                }
            }
        </script>
    </head>
    <body onLoad="inicializar()">
    <h1>Abrimos y cerramos ventanas</h1>
    <form>
        <input  type="button"  id="crear-ventana"  value="Crear Nueva Ventana">
        <input  type="button"  id="cerrar-ventana"  value="Cerrar Nueva Ventana">
    </form>
</html>
```

## Debes conocer

En los siguientes enlaces puedes ver varios ejemplos interactivos de la gestión de ventanas con el objeto **window**:

- Abrir y cerrar ventanas (w3school).

# 1.1.2.- Propiedades y métodos

El objeto **window** representa una ventana abierta en un navegador. Si un documento contiene marcos (**<frame>** o **<iframe>**), el navegador crea un objeto **window** para el documento HTML, y un objeto **window** adicional para para cada marco.



quinn.anya (CC BY-SA)

### Propiedades del objeto Window

| Propiedad | Descripción |
|---|---|
| **closed** | Devuelve un valor Boolean indicando cuando una ventana ha sido cerrada o no. |
| **defaultStatus** | Ajusta o devuelve el valor por defecto de la barra de estado de una ventana. |
| **document** | Devuelve el objeto **document** para la ventana. |
| **frames** | Devuelve un array de todos los marcos (incluidos iframes) de la ventana actual. |
| **history** | Devuelve el objeto **history** de la ventana. |
| **length** | Devuelve el número de frames (incluyendo iframes) que hay en dentro de una ventana. |
| **location** | Devuelve la Localización del objeto ventana (URL del fichero). |
| **name** | Ajusta o devuelve el nombre de una ventana. |
| **navigator** | Devuelve el objeto **navigator** de una ventana. |
| **opener** | Devuelve la referencia a la ventana que abrió la ventana actual. |
| **parent** | Devuelve la ventana padre de la ventana actual. |
| **self** | Devuelve la ventana actual. |
| **status** | Ajusta el texto de la barra de estado de una ventana. |

### Métodos del objeto Window

| Método | Descripción |
|---|---|
| **alert()** | Muestra una ventana emergente de alerta y un botón de aceptar. |
| **blur()** | Elimina el foco de la ventana actual. |
| **clearInterval()** | Resetea el cronómetro ajustado con **setInterval()**. |
| **setInterval()** | Llama a una función o evalúa una expresión en un intervalo especificado (en milisegundos). |
| **close()** | Cierra la ventana actual. |
| **confirm()** | Muestra una ventana emergente con un mensaje, un botón de aceptar y un botón de cancelar. |
| **focus()** | Coloca el foco en la ventana actual. |
| **open()** | Abre una nueva ventana de navegación. |
| **prompt()** | Muestra una ventana de diálogo para introducir datos. |

## Debes conocer

En los siguientes enlaces puedes ver varios ejemplos interactivos de la gestión de ventanas con el objeto **window**:

- Gestión de las dimensiones de las ventanas:
  - Acceso al tamaño interno de la ventana (w3school).
  - Acceso al tamaño externo de la ventana (w3school).
  - Aumentar el tamaño de una ventana en una cantidad con ResizeBy (w3School). Solo válido para ventanas abiertas con código JavaScript.
  - Establecer el tamaño de una ventana en una cantidad con ResizeTo (w3School). Solo válido para ventanas abiertas con código JavaScript.
- Gestión de la posición de las ventanas:
  - Mover la ventana una cantidad determinada con MoveBy (w3school). Solo válido para ventanas abiertas con código JavaScript.
  - Mover la ventana una posición determinada con MoveTo (w3school). Solo válido para ventanas abiertas con código JavaScript.
- Enfocar y desenfocar ventanas:
  - Enfocar una ventana (w3school).
  - Desenfocar una ventana (w3school).
- Gestión del scroll:
  - Posición del scroll (w3school).
  - Modificar el scroll en una cantidad con ScrollBy (w3school).
  - Establecer el scroll en una cantidad con ScrollTo (w3school).

# Autoevaluación

Para ir a la parte superior de la página se puede utilizar `window.scrollTo(0, 0);`

○ Verdadero ○ Falso

**Verdadero**

# 1.2.- Objeto location

El objeto **location** contiene información referente a la URL actual.

Este objeto, es parte del objeto **window** y accedemos a él a través de la propiedad **window.location**.

El objeto **location** contiene información referente a la URL actual.

Este objeto, es parte del objeto **window** y accedemos a él a través de la propiedad **window.location**.




chrisdlugosz (CC BY)

### Propiedades del objeto Location

| Propiedad | Descripción |
|---|---|
| **hash** | Cadena que contiene el nombre del enlace, dentro de la URL. |
| **host** | Cadena que contiene el nombre del servidor y el número del puerto, dentro de la URL. |
| **hostname** | Cadena que contiene el nombre de dominio del servidor (o la dirección IP), dentro de la URL. |
| **href** | Cadena que contiene la URL completa. |
| **pathname** | Cadena que contiene el camino al recurso, dentro de la URL. |
| **port** | Cadena que contiene el número de puerto del servidor, dentro de la URL. |
| **protocol** | Cadena que contiene el protocolo utilizado (incluyendo los dos puntos), dentro de la URL. |
| **search** | Cadena que contiene la información pasada en una llamada a un script, dentro de la URL. |

### Métodos del objeto Location

| | |
|---|---|
| **assign()** | Carga un nuevo documento. |
| **reload()** | Vuelve a cargar la URL especificada en la propiedad **href** del objeto **location**. |
| **replace()** | Reemplaza el historial actual mientras carga la URL especificada en cadenaURL. |

## Citas para pensar

"Mil rutas se apartan del fin elegido, pero hay una que llega a él."

***MONTAIGNE, Michel de.***

## Debes conocer

El siguiente enlace amplía información sobre el objeto **Location** y todas sus propiedades y métodos.

Más información y ejemplos sobre el objeto **Location.**

## Para saber más

La interfaz URL se usa para analizar, construir, normalizar y codificar las URL. Funciona proporcionando propiedades que le permiten leer y modificar fácilmente los componentes de una URL. Puedes profundizar en el siguiente enlace: https://developer.mozilla.org/es/docs/Web/API/URL

# 1.3.- Objeto navigator

Este objeto **navigator**, contiene información sobre el navegador que estamos utilizando cuando abrimos una URL o un documento local.




Caitlinator (CC BY)

## Propiedades del objeto Navigator

| Propiedad | Descripción |
|---|---|
| **appCodeName** | Cadena que contiene el nombre en código del navegador. |
| **appName** | Cadena que contiene el nombre del cliente. |
| **appVersion** | Cadena que contiene información sobre la versión del cliente. |
| **cookieEnabled** | Determina si las cookies están o no habilitadas en el navegador. |
| **platform** | Cadena con la plataforma sobre la que se está ejecutando el programa cliente. |
| **userAgent** | Cadena que contiene la cabecera completa del agente enviada en una petición HTTP. Contiene la información de las propiedades **appCodeName** y **appVersion**. |

## Métodos del objeto Navigator

| Método | Descripción |
|---|---|
| **javaEnabled()** | Devuelve true si el cliente permite la utilización de Java, en caso contrario, devuelve false. |

La propiedad **userAgent** permite conocer el equipo y navegador en el que se está ejecutando nuestra página web. Sin embargo, es necesario procesarla. Una posible forma es con las siguientes funciones:

```
function obtenerSistemaOperativo(userAgent) {
    if (/windows phone/i.test(userAgent)) {
        return "Windows Phone";
    }
    if (/win/i.test(userAgent)) {
        return "Windows";
    }
    if (/android/i.test(userAgent)) {
        return "Android";
    }
    if (/ipad|iphone|ipod/i.test(userAgent) && !/windows phone/i.test(userAgent)) {
        return "iOS";
    }
    if (/mac/i.test(userAgent)) {
        return "Mac OS";
    }
    if (/linux/i.test(userAgent)) {
        return "Linux";
    }
    return "Desconocido";
}

function obtenerNavegador(userAgent) {
    if (/chrome|crios|crmo/i.test(userAgent) && !/edg/i.test(userAgent)) {
        return "Chrome";
    }
    if (/firefox|fxios/i.test(userAgent)) {
```

```
        return "Firefox";
    }
    if (/safari/i.test(userAgent) && !/chrome|crios|crmo/i.test(userAgent)) {
        return "Safari";
    }
    if (/edg/i.test(userAgent)) {
        return "Edge";
    }
    if (/opr\//i.test(userAgent) || /opera/i.test(userAgent)) {
        return "Opera";
    }
    if (/msie|trident/i.test(userAgent)) {
        return "Internet Explorer";
    }
    return "Desconocido";
}
```

## Debes conocer

El siguiente enlace amplía información sobre el objeto **Navigator** y todas sus propiedades y métodos.

Más información y ejemplos sobre el objeto **Navigator**.

Detectar si se está en un dispositivo móvil o no.

## Autoevaluación

**Si queremos introducir datos a través de una ventana de diálogo en nuestra aplicación de JavaScript lo haremos con:**

- La propiedad **input** del objeto **window**.
- La propiedad **userAgent** del objeto **navigator**.
- El método **prompt** del objeto **window**.

No es correcta. No existe tal propiedad en el objeto **window**.

Incorrecta. Esa propiedad nos permite consultar información sobre el navegador utilizado.

Muy bien. Este método te permitirá solicitar datos al usuario, mediante una ventana de diálogo que podrás usar en tu aplicación.

### Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

## Para saber más

El objeto `history` proporciona acceso al historial del navegador. Este da acceso a métodos y propiedades útiles que permiten avanzar y retroceder a través del historial del usuario, así como manipular el contenido del historial.

- Propiedades y métodos del objeto History.
- Ejemplos interactivos del objeto History (w3school).

# 1.4.- Objeto document

El objeto **document** representa la página web, es decir, a la estructura de todos los elementos HTML y sus atributos (estos elementos). Habitualmente esta estructura es conocida como DOM y los elementos que lo componen son conocidos como nodos. Mediante el acceso y manipulación de los nodos que componen el DOM es posible hacer una página interactiva.



infilmity (CC BY-NC-SA)

En futuras unidades profundizaremos más sobre la manipulación del DOM. En esta nos centraremos en conocer un uso básico de este, accediendo a nodos y conociendo sus propiedades y métodos más sencillos.

El objeto **document** forma parte además del objeto **window**, y puede ser accedido a través de la propiedad **window.document** o directamente **document** (ya que podemos omitir la referencia a la **window** actual).

# 1.4.1.- Consulta de nodos del documento

Existen múltiples métodos para buscar nodos en el DOM en función del criterio de búsqueda que se quiera utilidad: id, clase, etiqueta, etc. Aunque en la actualidad es más habitual utilizar los métodos de propósito general, querySelector y querySelectorAll, que reciben como cadena un selector CSS y devuelven el nodo o lista de nodos que cumplan los criterios del selector.

Además, es importante tener en cuenta que algunos métodos de acceso devuelven un único nodo Node y otros lista de Nodos NodeList.

| | Acceso a nodo (clásico) | Acceso a nodo (con selector CSS) | Acceso a lista de nodos (clásico) | Acceso a lista de nodos (con selector CSS) |
|---|---|---|---|---|
| **Busca de un nodo por ID** | getElementById("id") | querySelector("#id") | - | querySelectorAll("#id") *uso no recomendado |
| **Busca de un nodo por clase** | - | querySelector(".clase") *devuelve solo el primero | getElementsByClassName("clase") | querySelectorAll(".clase") |
| **Busca de un nodo por etiqueta** | - | querySelector("etiqueta") *devuelve solo el primero | getElementsByTagName("etiqueta") | querySelectorAll("etiqueta") |

Uno de los errores de principiante más comunes es no conocer si un método de consulta devuelve un nodo o una lista de nodos. Por ejemplo, el siguiente código sería erroneo:

```
//Obtengo una lista de nodos
let nodos=document.getElementsByClassName("clase");

//Cambio el contenido del elemento (incorrecto)
nodos.innerHTML="Hola";

//Cambio el contenido del elemento (correcto)
for(let nodo of nodos) nodo.innerHTML="Hola";

// Cambio con arrayFunctions (correcto)
Array.from(nodos).forEach(nodo => {
    nodo.innerHTML = "Hola";
});
```

## Debes conocer

Es habitual hacer métodos atajo a querySelector con $ y a querySelectorAll con $$, siguiendo la nomenclatura de jQuery. Para ello basta con definir las siguientes funciones en nuestro proyecto:

```
function $(query) { return document.querySelector(query);}
function $$(query) { return document.querySelectorAll(query);}
```

## Debes conocer

El objeto **NodeList** no es un array tradicional y no cuenta con todas sus funcionalidades como las funciones de array como **map** o **reduce**. Para convertirlo a un array normal se puede utilizar el método **Array.from()**.

Por ejemplo, para obtener todos los nodos de la clase **page-header** y cambiarles el color de fondo a rojo se utiliza el siguiente código:

```
//Obtengo la lista de nodos
let listaDeNodos=document.querySelectorAll(".page-header");

//Convierto la lista a Array para poder utilizar las array functions.
listaDeNodos=Array.from(listaDeNodos);

//Utilizo la array function forEach
listaDeNodos.forEach(a => a.style.backgroundColor="red")
```

# 1.4.2.- Propiedades y métodos

Las principales **propiedades** y métodos del objeto document son:

### Colecciones del objeto Document ("accesos directos")

| Colección | Descripción |
|---|---|
| **anchors[]** | Es un array que contiene todos los hiperenlaces del documento. |
| **forms[]** | Es un array que contiene todos los formularios del documento. |
| **images[]** | Es un array que contiene todas las imágenes del documento. |
| **links[]** | Es un array que contiene todos los enlaces del documento. |

### Propiedades del objeto Document

| Propiedad | Descripción |
|---|---|
| **cookie** | Devuelve todos los nombres/valores de las cookies en el documento. |
| **domain** | Cadena que contiene el nombre de dominio del servidor que cargó el documento. |
| **referrer** | Cadena que contiene la URL del documento desde el cuál llegamos al documento actual. |
| **title** | Devuelve o ajusta el título del documento. |
| **URL** | Devuelve la URL completa del documento. |

### Propiedades del objeto Document

| Método | Descripción |
|---|---|
| **close()** | Cierra el flujo abierto previamente con **document.open()**. |
| **getElementById()** | Para acceder a un elemento identificado por el id escrito entre paréntesis. |
| **getElementsByName()** | Para acceder a los elementos identificados por el atributo **name** escrito entre paréntesis. |
| **getElementsByTagName()** | Para acceder a los elementos identificados por el tag o la etiqueta escrita entre paréntesis. |
| **open()** | Abre el flujo de escritura para poder utilizar **document.write()** o **document.writeln** en el documento. |
| **write()** | Para poder escribir expresiones HTML o código de JavaScript dentro de un documento. |
| **writeln()** | Lo mismo que **write()** pero añade un salto de línea al final de cada instrucción. |

## Debes conocer

El siguiente enlace amplía información sobre el objeto **Document** todas sus propiedades y métodos.

Más información y ejemplos sobre el objeto **Document.**

## Autoevaluación

¿Cuales de las siguientes instrucciones permiten acceder a un nodo por id?

- [ ] document.getElementById("id");
- [ ] document.querySelector("id");
- [ ] document.querySelector("#id");
- [ ] document.querySelector(".id");

Mostrar retroalimentación

# Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Incorrecto

# 2.- Objetos nativos en Javascript

## Caso práctico



Ministerio de Educación (Uso educativo nc)

El lenguaje JavaScript es un lenguaje basado en objetos. A **Antonio** le suena un poco el tema de objetos aunque nunca trabajó intensivamente con ellos. Como todos los lenguajes que incorporan sus funciones para realizar acciones, conversiones, etc., JavaScript también dispone de unos objetos nativos que le van a permitir a **Antonio** el realizar muchas de esas tareas.

Estos objetos, hacen referencia al trabajo con cadenas de texto, operaciones matemáticas, números, valores booleanos y trabajo con fechas y horas.

Ésto le va a ser muy útil para realizar su aplicación ya que tendrá que realizar diferentes tipos de conversiones de datos, trabajar intensivamente con cadenas y por supuesto con fechas y horas.

Aunque no hemos visto como crear objetos, sí que ya hemos dado unas pinceladas a lo que son los objetos, propiedades y métodos.

En esta sección vamos a echar una ojeada a objetos que son nativos en JavaScript, ésto es, aquello que JavaScript nos da, listos para su utilización en nuestra aplicación.

Echaremos un vistazo a los objetos **String**, **Math**, **Number**, **Boolean** y **Date**.

## Citas para pensar

"Si me hubieran hecho objeto sería objetivo, pero me hicieron sujeto."

***BERGAMÍN, José.***

## Reflexiona

¿Te has parado a pensar alguna vez que nuestro mundo está rodeado de objetos por todas partes?

¿Sabes que prácticamente, todos esos objetos tienen algunas propiedades como pueden ser tamaño, color, peso, tipo de corriente que usan, temperatura, tipo de combustible, etc.?

¿Sabes que también podemos realizar acciones con esos objetos, como pueden ser encender, apagar, mover, abrir, cerrar, subir temperatura, bajar temperatura, marcar número, colgar, etc.?

# 2.1.- Objeto String



faccig (CC BY-SA)

Una cadena (string) consta de uno o más caracteres de texto, rodeados de comillas simples o dobles; da igual cuales usemos ya que se considerará una cadena de todas formas, pero en algunos casos resulta más cómodo el uso de unas u otras.

Recuerda que de cómo definir las cadenas ya hemos hablado y que no se te olviden las **plantillas de cadenas** que te hacen muy fácil su composición en vez de estar concatenando que a veces resulta engorroso.

**Caracteres especiales o caracteres de escape.**

La forma en la que se crean las cadenas en JavaScript, hace que cuando tengamos que emplear ciertos caracteres especiales en una cadena de texto, tengamos que escaparlos, empleando el símbolo \ seguido del carácter.

Vemos aquí un listado de los caracteres especiales o de escape en JavaScript:

### Caracteres de escape y especiales en JavaScript

| Símbolos | Explicación | Símbolos | Explicación |
|---|---|---|---|
| \" | Comillas dobles | \t | Tabulador |
| \' | Comilla simple | \n | Nueva línea |
| \\ | Barra inclinada | \r | Salto de línea |
| \b | Retroceso | \f | Avance de página |

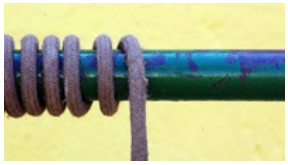
## Debes conocer

El siguiente enlace amplía información sobre el objeto `String` y todas sus propiedades y métodos.

Más información y ejemplos sobre el objeto `String`.

# 2.1.1.- Propiedades y métodos del objeto String

Para crear un objeto **String** lo podremos hacer de la ssiguientes formas:



Josep Ma. Rosell (CC BY)

```
let cadena1 = new String('Primera cadena');
let cadena2 = "Segunda cadena";
let cadena3 = 'Tercera cadena';
let todasCadenas = `${cadena1} - ${cadena2} - ${cadena3}`;
```

Es decir, cada vez que tengamos una cadena de texto, en realidad es un objeto **String** que tiene propiedades y métodos:

```
cadena.propiedad;
cadena.metodo( [parámetros] );
```

### Propiedades del objeto String

| Propiedad | Descripción |
|---|---|
| **length** | Contiene la longitud de una cadena. |

### Métodos del objeto String

| Métodos | Descripción |
|---|---|
| **charAt()** | Devuelve el carácter especificado por la posición que se indica entre paréntesis. |
| **charCodeAt()** | Devuelve el Unicode del carácter especificado por la posición que se indica entre paréntesis. |
| **concat()** | Une una o más cadenas y devuelve el resultado de esa unión. |
| **fromCharCode()** | Convierte valores Unicode a caracteres. |
| **indexOf()** | Devuelve la posición de la primera ocurrencia del carácter buscado en la cadena. |
| **lastIndexOf()** | Devuelve la posición de la última ocurrencia del carácter buscado en la cadena. |
| **match()** | Busca una coincidencia entre una expresión regular y una cadena y devuelve las coincidencias o **null** si no ha encontrado nada. |
| **replace()** | Busca una subcadena en la cadena y la reemplaza por la nueva cadena especificada. |
| **search()** | Busca una subcadena en la cadena y devuelve la posición dónde se encontró. |
| **slice()** | Extrae una parte de la cadena y devuelve una nueva cadena. |
| **split()** | Divide una cadena en un array de subcadenas. |
| **substr()** | Extrae los caracteres de una cadena, comenzando en una determinada posición y con el número de caracteres indicado. |
| **substring()** | Extrae los caracteres de una cadena entre dos índices especificados. |
| **toLowerCase()** | Convierte una cadena en minúsculas. |
| **toUpperCase()** | Convierte una cadena en mayúsculas. |

## Autoevaluación

Cada vez que tenemos una cadena de texto en JavaScript, en realidad estamos trabajando con un objeto de tipo String que tiene propiedades y métodos.

○ Verdadero ○ Falso

**Verdadero**

En JavaScript, las cadenas de texto son objetos `String` que incluyen propiedades y métodos, como `length` para obtener la longitud de la cadena.

# 2.2.- Objeto Math

Ya vimos anteriormente algunas funciones, que nos permitían convertir cadenas a diferentes formatos numéricos (**parseInt**, **parseFloat**). A parte de esas funciones, disponemos de un objeto **Math** en JavaScript, que nos permite realizar operaciones matemáticas. El objeto **Math** no posee un constructor (no nos permitirá por lo tanto crear o instanciar nuevos objetos que sean de tipo **Math**), por lo que para llamar a sus propiedades y métodos, lo haremos anteponiendo **Math** a la propiedad o el método. Por ejemplo:



conskeptical (CC BY-SA)

```
let x = Math.PI;            // Devuelve el número PI.
let y = Math.sqrt(16);      // Devuelve la raíz cuadrada de 16.
```

### Propiedades del objeto Math

| Propiedad | Descripción |
|---|---|
| **E** | Devuelve el número Euler (aproximadamente 2.718). |
| **LN2** | Devuelve el logaritmo neperiano de 2 ( aproximadamente 0.693). |
| **LN10** | Devuelve el logaritmo neperiano de 10 ( aproximadamente 2.302). |
| **LOG2E** | Devuelve el logaritmo base 2 de E ( aproximadamente 1.442). |
| **LOG10E** | Devuelve el logaritmo base 10 de E ( aproximadamente 0.434). |
| **PI** | Devuelve el número PI ( aproximadamente 3.14159). |
| **SQRT2** | Devuelve la raíz cuadrada de 2 ( aproximadamente 1.414). |

### Métodos del objeto Math

| Método | Descripción |
|---|---|
| **abs(x)** | Devuelve el valor absoluto de x. |
| **acos(x)** | Devuelve el arcocoseno de x, en radianes. |
| **asin(x)** | Devuelve el arcoseno de x, en radianes. |
| **atan(x)** | Devuelve el arcotangente de x, en radianes con un valor entre -PI/2 y PI/2. |
| **atan2(y,x)** | Devuelve el arcotangente del cociente de sus argumentos. |
| **ceil(x)** | Devuelve el número x redondeado al alta hacia el siguiente entero. |
| **cos(x)** | Devuelve el coseno de x (x está en radianes). |
| **floor(x)** | Devuelve el número x redondeado a la baja hacia el anterior entero. |
| **log(x)** | Devuelve el logaritmo neperiando (base E) de x. |
| **max(x,y,z,...,n)** | Devuelve el número más alto de los que se pasan como parámetros. |
| **min(x,y,z,...,n)** | Devuelve el número más bajo de los que se pasan como parámetros. |
| **pow(x,y)** | Devuelve el resultado de x elevado a y. |
| **random()** | Devuelve un número al azar entre 0 y 1. |
| **round(x)** | Redondea x al entero más próximo. |
| **sin(x)** | Devuelve el seno de x (x está en radianes). |

| Método | Descripción |
| --- | --- |
| sqrt(x) | Devuelve la raíz cuadrada de x. |
| tan(x) | Devuelve la tangente de un ángulo. |

## Para saber más

Ejemplo de mal uso del objeto Math para crear urgencia en el usuario.

# 2.3.- Objeto Number



Darwin Bell (CC BY)

El objeto **Number** se usa muy raramente, ya que para la mayor parte de los casos, JavaScript satisface las necesidades del día a día con los valores numéricos que almacenamos en variables. Pero el objeto **Number** contiene alguna información y capacidades muy interesantes para programadores más serios.

Lo primero, es que el objeto **Number** contiene propiedades que nos indican el rango de números soportados en el lenguaje. El número más alto es **1.79E + 308**; el número más bajo es **2.22E-308**. Cualquier número mayor que el número más alto, será considerado como infinito positivo, y si es más pequeño que el número más bajo, será considerado infinito negativo.

Los números y sus valores están definidos internamente en JavaScript, como valores de doble precisión y de 64 bits.

El objeto **Number**, es un objeto envoltorio para valores numéricos primitivos.

Los objetos **Number** son creados con **new Number()**.

### Propiedades del objeto Number

| Propiedad | Descripción |
|---|---|
| **constructor** | Devuelve la función que creó el objeto **Number**. |
| **MAX_VALUE** | Devuelve el número más alto disponible en JavaScript. |
| **MIN_VALUE** | Devuelve el número más pequeño disponible en JavaScript. |
| **NEGATIVE_INFINITY** | Representa a infinito negativo (se devuelve en caso de overflow). |
| **POSITIVE_INFINITY** | Representa a infinito positivo (se devuelve en caso de overflow). |
| **prototype** | Permite añadir nuestras propias propiedades y métodos a un objeto. |

### Métodos del objeto Number

| | |
|---|---|
| **toExponential(x)** | Convierte un número a su notación exponencial. |
| **toFixed(x)** | Formatea un número con x digitos decimales después del punto decimal. |
| **toPrecision(x)** | Formatea un número a la longitud x. |
| **toString()** | Convierte un objeto **Number** en una cadena.<br>- Si se pone 2 como parámetro se mostrará el número en binario.<br>- Si se pone 8 como parámetro se mostrará el número en octal.<br>- Si se pone 16 como parámetro se mostrará el número en hexadecimal. |
| **valueOf()** | Devuelve el valor primitivo de un objeto **Number**. |

# 2.4.- Objeto Boolean

El objeto **Boolean** se utiliza para convertir un valor no lógico, a un valor lógico (**true** o **false**).



Everaldo Coelho (GNU/GPL)

### Propiedades del objeto Boolean

| | |
|---|---|
| **constructor** | Devuelve la función que creó el objeto **Boolean**. |
| **prototype** | Te permitirá añadir propiedades y métodos a un objeto. |

### Métodos del objeto Boolean

| | |
|---|---|
| **toString()** | Convierte un valor **Boolean** a una cadena y devuelve el resultado. |
| **valueOf()** | Devuelve el valor primitivo de un objeto **Boolean**. |

## Para saber más

En el siguiente enlace podrás encontrar ejemplos sobre el uso del objeto **Boolean**.

Más información y ejemplos sobre el objeto **Boolean**.

## Autoevaluación

**¿Para usar un objeto Math deberemos instanciarlo antes de poder usarlo?**

- ○ Sí.
- ○ No.

No es correcto porque este objeto no dispone de constructor, con lo que para usarlo lo llamaremos directamente seguido de su propiedad o método.

Es correcto porque este objeto no dispone de constructor con lo que no podremos crear instancias del mismo.

### Solución

1. Incorrecto
2. Opción correcta

# 2.5.- Objeto Date

El objeto **Date** se utiliza para trabajar con fechas y horas. Los objetos **Date** se crean con **new Date()**.



Everaldo Coelho (GNU/GPL)

Hay 4 formas de instanciar (crear un objeto de tipo **Date**):

```
let dia1 = new Date();
let dia2 = new Date(milisegundos);
let dia3 = new Date(cadena de Fecha);
let dia4 = new Date(año, mes, día, horas, minutos, segundos, milisegundos);
```

### Propiedades del objeto Date

| Propiedad | Descripción |
|---|---|
| **constructor** | Devuelve la función que creó el objeto **Date**. |
| **prototype** | Te permitirá añadir propiedades y métodos a un objeto. |

### Métodos del objeto Date

| | |
|---|---|
| **getDate()** | Devuelve el día del mes (de 1-31). |
| **getDay()** | Devuelve el día de la semana (de 0-6). |
| **getFullYear()** | Devuelve el año (4 dígitos). |
| **getHours()** | Devuelve la hora (de 0-23). |
| **getMilliseconds()** | Devuelve los milisegundos (de 0-999). |
| **getMinutes()** | Devuelve los minutos (de 0-59). |
| **getMonth()** | Devuelve el mes (de 0-11). |
| **getSeconds()** | Devuelve los segundos (de 0-59). |
| **getTime()** | Devuelve los milisegundos desde media noche del 1 de Enero de 1970. |
| **getTimezoneOffset()** | Devuelve la diferencia de tiempo entre GMT y la hora local, en minutos. |
| **getUTCDate()** | Devuelve el día del mes en base a la hora UTC (de 1-31). |
| **getUTCDay()** | Devuelve el día de la semana en base a la hora UTC (de 0-6). |
| **getUTCFullYear()** | Devuelve el año en base a la hora UTC (4 dígitos). |
| **setDate()** | Ajusta el día del mes del objeto (de 1-31). |
| **setFullYear()** | Ajusta el año del objeto (4 dígitos). |
| **setHours()** | Ajusta la hora del objeto (de 0-23). |

## Debes conocer

Para asignar una fecha a un campo de texto de tipo date se debe hacer uso de la propiedad valueAsDate.

```
myInput.valueAsDate = new Date(2024,0,1);
```

# 3.- Interacción y control de contenido

## Caso práctico

Bajo la tutoría de **Ana, Antonio** se dispone a desarrollar una página web interactiva que permitirá a los usuarios de una tienda en línea filtrar productos y visualizar detalles de manera dinámica, utilizando JavaScript.

Este ejercicio le permitirá a **Antonio** aplicar conceptos avanzados de JavaScript, centrados en la manipulación del DOM y la interacción del usuario, habilidades que son fundamentales para el desarrollo de aplicaciones web modernas.



Ministerio de Educación (Uso educativo nc)

La interacción y el control del contenido en una página web son esenciales para crear experiencias dinámicas y atractivas para los usuarios. JavaScript nos permite responder a las acciones del usuario y actualizar el contenido de la página de manera eficiente.

En esta sección, aprenderemos a realizar una interacción con el usuario simple con una primera aproximación a los eventos en JavaScript, así como a revisar cómo hacer uso de temporizadores o cómo integrar contenido externo.

# 3.1.- Interacción de usuario

La interacción de usuario en JavaScript se maneja principalmente a través de eventos. Los eventos son acciones que ocurren en la página, como hacer clic en un botón, mover el ratón, o presionar una tecla. Para responder a estos eventos, utilizamos `addEventListener` para adjuntar una función que se ejecutará cuando el evento ocurra. Los eventos son un elemento fundamental de JavaScript y seguiremos profundizando sobre ellos a lo largo del curso (por ejemplo, las unidad 5 está dedicada principalmente a la gestión de eventos).

A continuación, veremos un ejemplo básico en el que cambiaremos el texto de un párrafo al hacer clic en un botón.

## HTML

Tenemos un botón con el id changeTextButton y un párrafo con el id message.

```
<div class="container">
    <h2>Cambiar Texto con un Botón</h2>
    <button id="changeTextButton">Haz clic aquí</button>
    <p id="message">Texto original</p>
</div>
```

## CSS

Estilos básicos para una mejor presentación.

```
body {
    font-family: Arial, sans-serif;
    margin: 20px;
}
.container {
    max-width: 600px;
    margin: auto;
    text-align: center;
}
#message {
    margin-top: 20px;
    font-size: 1.2em;
}
```

## JavaScript

Al cargar el documento (DOMContentLoaded), se ejecuta la función init que configura la referencia a los elementos del DOM y añade el event listener al botón. Por último, al hacer clic en el botón, se ejecuta una función anónima que cambia el texto del párrafo.

```
// Inicialización
document.addEventListener("DOMContentLoaded", init);

function init() {
    // Referencia a los elementos del DOM
    const changeTextButton = document.getElementById("changeTextButton");
    const message = document.getElementById("message");

    // Event Listener para el botón
```

```
    changeTextButton.addEventListener("click", function() {
        // Cambia el texto del párrafo
        message.textContent = "¡El texto ha cambiado!";
    });
}
```

## Debes conocer

A continuación tienes más ejemplos de interacciones simples de usuario:

- Interacciones simples (w3shool).
- Modificación de código CSS (w3school).
- Interacción por consola y ventanas emergentes.

## Autoevaluación

Un evento en JavaScript es una acción que ocurre en la página, como hacer clic en un botón, y permite ejecutar una función en respuesta a esa acción.

○ Verdadero ○ Falso

**Verdadero**

# 3.2.- Temporizadores

Los temporizadores permiten ejecutar una función en un tiempo determinado, con setTimeout(funcion, tiempoEnMilisegundos), o que una función se ejecute repetidamente cada cierto tiempo, con setInterval(funcion, tiempoEnMilisegundos). También se puede detener la ejecución de un setTimeout o SetInterval utilizando el ID que devuelve con clearTimeout(id) ó clearInterval(id).

Vamos a hacer un ejemplo sencillo de una web que realice la cuenta atrás hasta final de año:

## HTML

```html
<html>
    <head>
        <meta charset="utf-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <title></title>
        <meta name="description" content="">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <link rel="stylesheet" href="">
    </head>
    <body>
        <h2 id="date"></h2>

        <script src="main.js" async defer></script>
    </body>
</html>
```

## JavaScript

```javascript
//Variables
let tiempoRestanteNodo; //Referencia al nodo del DOM con ID date
let fechaLimite; //Fecha de fin de año del año actual

//Inicialización
window.addEventListener("load",init);

function init(){
    //Se obtienen las referencias a los nodos del DOM
    tiempoRestanteNodo=document.getElementById("date");

    //Se obtiene la fecha de fin de año del año actual
    fechaLimite = new Date(`December 31, ${new Date().getFullYear()} 23:59:59`);

    //Se activa el temporizador, cada segundo (o 1000 milisegundos)
    setInterval(update,1000);
}

//Metodo que se ejecuta cada segundo
function update(){
    //Se obtiene la fecha actual
    let fechaActual=new Date();

    //Se calcula la diferencia entre la fecha de fin de año y la fecha actual y se construye una cadena
    let textoDi=`
    Meses: ${fechaLimite.getMonth()-fechaActual.getMonth()};
    Dias ${fechaLimite.getDate()-fechaActual.getDate()};
    Horas ${fechaLimite.getHours()-fechaActual.getHours()};
    Minutos ${fechaLimite.getMinutes()-fechaActual.getMinutes()};
    Segundos ${fechaLimite.getSeconds()-fechaActual.getSeconds()}`;

    //Se actualiza la página con la nueva fecha
```

```
    tiempoRestanteNodo.innerHTML=textoDi;
}
```

# 3.3.- Iframes

Tradicionalmente, si se quería insertar una página dentro de otra se utilizaban frameset y frames. Sin embargo, en la actualidad estos se encuentran obsoletos y no se recomienda su utilización.

En la actualidad, en su lugar se hace uso de los iFrames. Un iframe es un elemento HTML que permite incrustar otra página HTML dentro de la página actual. Los iframes son más versátiles y permiten una mejor integración con el contenido del sitio principal. A continuación se presenta un ejemplo básico de cómo utilizar un iframe:

```
<h1>Página Principal</h1>
<iframe src="https://www.ejemplo.com" width="600" height="400" frameborder="0">
    Tu navegador no soporta iframes.
</iframe>
```

Algunos ejemplos tipicos de iFrames es cuando se inserta un video desde Youtube o un mapa de Google Maps:

## Video (código)

```
<iframe width="300" height="169" src="https://www.youtube.com/embed/dQw4w9WgXcQ?si=VCX7wIGM-dyxY_b7" title="YouTube video p
```

## Video (ejemplo)

https://www.youtube.com/embed/dQw4w9WgXcQ?si=VCX7wIGM-dyxY_b7

## Mapa (código)

```
<iframe src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3192.679437299913!2d-2.467697524491662!3d36.850152664882
```

## Mapa (ejemplo)

https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3192.679437299913!2d-2.467697524491662!3d36.85015266488263!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0xd7075f9253c8cfd%3A0x10423ef17d0655c!2sI.E.S.%20Al-%C3%81ndalus!5e0!3m2!1ses!2ses!4v1719400903359!5m2!1ses!2ses

## Debes conocer

Cuando se hace uso de iFrames se está realizando un intercambio de recursos entre diferentes orígenes (páginas). Para que esto sea posible es necesario que ambos sitios lo permitan en la configuración de su CORS (Cross-Origin

Resource Sharing). CORS es un mecanismo de seguridad que permite controlar cómo los recursos de un servidor pueden ser solicitados desde otro dominio. Para permitir que un sitio se muestre en un iframe de otro origen, el servidor de la página que se va a incrustar debe incluir las cabeceras adecuadas en su respuesta HTTP, como:

```
Access-Control-Allow-Origin: https://www.otro-ejemplo.com
Access-Control-Allow-Methods: GET
```

Es importante mencionar que el uso de iframes también tiene sus limitaciones y posibles problemas de seguridad, como los ataques de clickjacking. Por esta razón, es crucial asegurarse de implementar medidas de seguridad adecuadas, como la cabecera HTTP X-Frame-Options que indica a los navegadores si deben o no permitir que una página se muestre dentro de un iframe. Por ejemplo:

```
X-Frame-Options: DENY // Opción 1

X-Frame-Options: SAMEORIGIN // Opción 2
```

Estas configuraciones ayudan a proteger tu sitio web contra intentos no autorizados de incrustar tu contenido en otros sitios.

# 4. Almacenamiento y recuperación de información

## Caso práctico



Ministerio de Educación (Uso educativo nc)

**Antonio** está preparado para profundizar en los métodos de almacenamiento y recuperación de información en aplicaciones web.

A medida que **Antonio** avanza en su aprendizaje, se le ha encomendado una tarea fundamental: utilizar las diferentes opciones de almacenamiento en el lado del cliente (Cookies, Local Storage y Session Storage) para gestionar datos de usuario en una aplicación web.

En el desarrollo web, es crucial poder almacenar y recuperar información de manera eficiente y segura en el navegador del usuario. JavaScript proporciona varias herramientas y APIs para manejar esta tarea. En esta sección, abordaremos las principales tipos de almacenamiento en el lado del cliente: cookies, local storage y session storage.

Todos estos tipos permiten almacenar y recuperar información, pero tienen varias diferencias entre sí:

| | **Cookies.** | **Local Storage.** | **Session Storage.** |
|---|---|---|---|
| **Capacidad.** | 4 KB. | 10 MB. | 5 MB. |
| **Accesible desde.** | Cualquier ventana y pestaña. | Cualquier ventana y pestaña. | Desde la pestaña en la que se creó. |
| **Caducidad.** | Establecido manualmente al crear la cookie. | Nunca. | Al cerrar la pestaña. |
| **Lugar almacenamiento.** | Servidor y navegador cliente. | Navegador cliente | Navegador cliente. |
| **Enviado con las peticiones.** | Sí. | No. | No. |

Todos estos métodos de almacenamiento manejan la información en formato de texto. Por lo tanto, es fundamental explorar cómo utilizar JSON para parsear y almacenar objetos de manera eficiente.

## Autoevaluación

La información almacenada en Local Storage se borra al cerrar la pestaña.

○ Verdadero ○ Falso

**Falso**

La información guardada en Local Storage nunca se borra, a diferencia de la almacenada en Session Storage que se borra al cerrar la pestaña.

# 4.1.- Cookies

Las cookies son pequeños fragmentos de datos almacenados en el navegador del usuario por el sitio web que está siendo visitado. Se utilizan principalmente para recordar información entre diferentes visitas o sesiones de navegación. En JavaScript, se pueden manipular las cookies a través del objeto **document.cookie**.

El valor de **document.cookie** es una cadena especial con el siguiente formato:

```
"variable=valor;expires=fecha expiración;path=/"
```

Donde

- variable es el nombre de la cookie.
- valor es el valor de la cookie.
- expires es la fecha de expiración de la cookie (la forma de borrar una cookie es cambiar su fecha de expiración a una pasada).
- path es el lugar del dominio donde la cookie es válida.

Puedes modificar el valor de las cookies directamente mediante la asignación a **document.cookie**. Por ejemplo:

```
document.cookie = "username=Oscar Gomez; expires=Fri, 28 Jun 2024 12:00:00 UTC; path=/"
```

Sin embargo, según las cookies se complican, la gestión de esta cadena no es tan trivial y se recomienda utilizar las siguientes funciones de apoyo para interactuar con ella

```
//Función para establecer una cookie
function setCookie(cname,cvalue,exdays) {
  const d = new Date();
  d.setTime(d.getTime() + (exdays*24*60*60*1000));
  let expires = "expires=" + d.toUTCString();
  document.cookie = cname + "=" + cvalue + ";" + expires + ";path=/";
}

//Función para recuperar el valor de una cookie
function getCookie(cname) {
  let name = cname + "=";
  let decodedCookie = decodeURIComponent(document.cookie);
  let ca = decodedCookie.split(';');
  for(let i = 0; i < ca.length; i++) {
    let c = ca[i];
    while (c.charAt(0) == ' ') {
      c = c.substring(1);
    }
    if (c.indexOf(name) == 0) {
      return c.substring(name.length, c.length);
    }
  }
  return "";
}
```

Por ejemplo, una forma de utilizarlas sería:

```
function checkCookie() {
  let user = getCookie("username");
  if (user != "") {
    alert("Welcome again " + user);
  } else {
     user = prompt("Please enter your name:","");
     if (user != "" && user != null) {
       setCookie("username", user, 30);
     }
  }
}
```

En este ejemplo, la función checkCookie verifica si la cookie "username" existe. Si existe, muestra un mensaje de bienvenida. Si no existe, pide al usuario que ingrese su nombre y establece una cookie con este valor que expira en 30 días. Estas herramientas

y métodos facilitan la gestión de las cookies, mejorando la experiencia de usuario y manteniendo la información persistente de manera efectiva en las aplicaciones web.

## Para saber más

Puedes profundizar más sobre el uso de cookies con JavaScript en: https://www.w3schools.com/js/js_cookies.asp

# 4.2.- LocalStorage y SessionStorage

Los objetos localStorage y sessionStorage permiten guardar pares de clave/valor en el navegador. A diferencia de las cookies, no se envían al servidor en cada petición, por lo que permiten almacenar mucha más información. Diferenciándose ambos principalmente en la persistencia de la información, con localStorage la información nunca expira y con sessionStorage la información se pierde al cerrar la pestaña actual.

Ambos objetos proporcionan los mismos métodos y propiedades:

- setItem(clave, valor) – almacenar un par clave/valor.
- getItem(clave) – obtener el valor por medio de la clave.
- removeItem(clave) – eliminar la clave y su valor.
- clear() – borrar todo.

También es posible acceder a la información como si fueran parámetros del objeto `storage`:

```
if (localStorage.clickcount) {
  localStorage.clickcount = Number(localStorage.clickcount) + 1;
} else {
  localStorage.clickcount = 1;
}
```

## Número de *clicks*

Vamos a hacer un ejemplo simple que lleve el número de clicks que ha realizado el usuario en la página y mostrará el valor por consola:

```
window.addEventListener("click",nuevoClick);

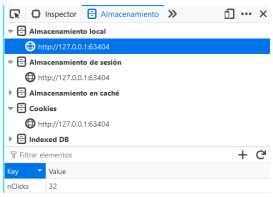
function nuevoClick(){
    //1. Recuperamos el número de clicks actual. Si no hubiera valor, inicializamos a 0
    let nClicks = localStorage.getItem("nClicks") ?? 0;

    //2. Aumentamos el número de clicks y lo mostramos por consola
    nClicks++;
    console.log(`Número de clicks: ${nClicks}`);

    //3. Guardamos el número de clicks en memoria
    localStorage.setItem("nClicks",nClicks);

}
```

Las claves y valores almacenados en memoria local pueden ser consultados mediante las herramientas del desarrollador (F12), Almacenamiento.




Óscar Gómez. *Herramientas del desarrollador. Almacenamiento.* (Dominio público)

En este ejemplo, al utilizar localStorage, la información se mantiene incluso si se cierra el navegador.

## Para saber más

Puedes profundizar sobre los objetos LocalStorage y SessionStorage en:

- https://es.javascript.info/localstorage

## Cambiar esquema de colores (con clases)

Vamos a hacer un segundo ejemplo. En este deseamos almacenar si el usuario utiliza modo claro u oscuro e inicializar la página según el valor almacenado. Utilizándose el modo claro por defecto.

### HTML

```
<!DOCTYPE html>
<!--[if lt IE 7]>      <html class="no-js lt-ie9 lt-ie8 lt-ie7"> <![endif]-->
<!--[if IE 7]>         <html class="no-js lt-ie9 lt-ie8"> <![endif]-->
<!--[if IE 8]>         <html class="no-js lt-ie9"> <![endif]-->
<!--[if gt IE 8]>      <html class="no-js"> <!--<![endif]-->
<html>
    <head>
        <meta charset="utf-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <title></title>
        <meta name="description" content="">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <link rel="stylesheet" href="./css/style.css">
    </head>
    <body>

        <h1>¡Hola Storage!</h1>
        <input type="button" id="changeMode" value="Cambiar modo" />
        <script src="./js/main.js" async defer></script>
    </body>
</html>
```

### CSS

```
body{
    width: 100%;
    max-width: 1000px;
    margin: auto;

    text-align: center;

    background-color: cyan;
    color: black;

}

.darkMode{
    background-color: darkcyan;
    color: white;
```

```
}
```

## JavaScript

```
// Inicialización
window.addEventListener("load",init);

function init() {
    // 1. Activamos el modo oscuro, si fue el último que se utilizó
    // Verificamos si el modo almacenado en localStorage es "dark"
    if (localStorage.getItem("mode") == "dark") {
        // Si es "dark", cambiamos el modo a oscuro
        toggleMode();
    }

    // 2. Asociamos el evento de cambiar modo al botón
    // Obtenemos el botón con el id "changeMode" y le asociamos el evento de click para cambiar el modo
    document.querySelector("#changeMode").addEventListener("click", toggleMode);
}

function toggleMode() {
    // Alternamos la clase "darkMode" en el cuerpo del documento
    document.body.classList.toggle("darkMode");

    // Guardamos el modo actual en localStorage
    // Si el cuerpo del documento tiene la clase "darkMode", almacenamos "dark", de lo contrario almacenamos "light"
    localStorage.setItem("mode", document.body.classList.contains("darkMode") ? "dark" : "light");
}
```

## Para saber más

Si en el navegador se establece un tema de colores oscuros o claros, es posible crear reglas CSS (media query) que muestren colores diferentes en función del valor de la variable prefers-color-scheme. /* Estilos por defecto (por ejemplo, tema claro) */

```
body {
    background-color: white;
    color: black;
}

/* Estilos para tema oscuro */
@media (prefers-color-scheme: dark) {
    body {
        background-color: black;
        color: white;
    }
}
```

# 4.3.- Serialización a JSON

Los métodos de almacenamiento vistos (cookies, localStorage y sessionStorage) solo permiten almacenar cadenas de texto. Para almacenar otros tipos de datos, es necesario convertirlos antes de exportarlos e importarlos. En el caso de los números, como en el ejemplo de un contador, esta tarea es trivial, ya que las conversiones se realizan implícitamente durante las operaciones. Sin embargo, este no es siempre el caso con otros tipos de datos.

JSON permite representar objetos complejos en un formato de texto estructurado, que es fácil de leer y escribir tanto para humanos como para máquinas. Cuando utilizamos JSON para almacenar datos, primero convertimos nuestros objetos (como arrays, diccionarios u otras estructuras de datos) en cadenas JSON. Este proceso se conoce como serialización. La serialización toma los datos en su formato nativo y los convierte en una cadena de texto JSON.

Por ejemplo, si queremos guardar un array en localStorage, primero debemos convertirlo a una cadena de texto. Esto se puede lograr utilizando el método JSON.stringify(). Al recuperar el array, debemos realizar la conversión inversa, utilizando el método JSON.parse(), para obtener nuevamente el array original. Este proceso de serialización y deserialización permite almacenar y recuperar estructuras de datos complejas en localStorage, manteniendo su integridad y estructura original.

Aquí hay un ejemplo práctico de cómo almacenar y recuperar un array en localStorage:

```
// Ejemplo de almacenamiento de un array en localStorage
let miArray = [1, 2, 3, 4, 5];

// Convertir el array a una cadena de texto y almacenarlo en localStorage
localStorage.setItem('miArray', JSON.stringify(miArray));

// Recuperar la cadena de texto desde localStorage y convertirla nuevamente a un array
let arrayRecuperado = JSON.parse(localStorage.getItem('miArray'));

console.log(arrayRecuperado); // [1, 2, 3, 4, 5]
```

## Debes conocer

Puedes visualizar la estructura de cualquier objecto JSON con https://www.toptal.com/developers/json-formatter

## Para saber más

Vamos a aplicar este ejemplo para guardar/cargar automáticamente el contenido de todos los inputs de una página. Este código es algo complejo, por lo que revísalo por calma y no tengas vergüenza en preguntar

### HTML

```
<!DOCTYPE html>
<!--[if lt IE 7]>      <html class="no-js lt-ie9 lt-ie8 lt-ie7"> <![endif]-->
<!--[if IE 7]>         <html class="no-js lt-ie9 lt-ie8"> <![endif]-->
<!--[if IE 8]>         <html class="no-js lt-ie9"> <![endif]-->
<!--[if gt IE 8]>      <html class="no-js"> <!--<![endif]-->
<html>
    <head>
        <meta charset="utf-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <title></title>
        <meta name="description" content="">
        <meta name="viewport" content="width=device-width, initial-scale=1">
    </head>
```

```
    <body>
        <h1>¡Hola Storage + JSON!</h1>

        <!-- Se pueden añadir tantos inputs como se desee -->
        <input type="text" value="1"/>
        <input type="text" value="2"/>

        <script src="./js/main.js" async defer></script>
    </body>
</html>
```

## JavaScript

```
// Funciones de inicialización
window.addEventListener("load", init);

function init() {
    loadInputs();
    syncInputsChanges();
}

// Función de carga de inputs
function loadInputs() {
    const storedInputs = localStorage.getItem("inputs");

    // Si no hay datos guardados, terminamos
    if (!storedInputs) return;

    // Parseo la cadena para convertirla en un array
    const inputsValues = JSON.parse(storedInputs);

    // Obtengo los nodos de los inputs y establezco los valores
    const inputs = document.querySelectorAll("input");
    inputs.forEach((input, index) => {
        input.value = inputsValues[index] || '';
    });
}

// Función para subscribirse a cambios de los inputs
function syncInputsChanges() {
    const inputs = document.querySelectorAll("input");
    inputs.forEach(input => input.addEventListener("input", saveInputs));
}

// Función para guardar la información de los inputs
function saveInputs() {
    const inputs = Array.from(document.querySelectorAll("input"));
    const inputsValues = inputs.map(input => input.value);

    // Convierto el array a JSON y guardo en localStorage
    localStorage.setItem("inputs", JSON.stringify(inputsValues));
}
```

Adicionalmente, se puede detectar cuando un usuario cambia de pestaña y lanzar de nuevo la función **loadInputs()** utilizando el evento **visibilitychange**. Para ello basta con añadir la siguiente instrucción entres las funciones de inicialización.

```
document.addEventListener("visibilitychange",loadInputs);
```

# Autoevaluación

Para almacenar un array en `localStorage`, es necesario convertirlo en una cadena de texto utilizando `JSON.stringify()` y, posteriormente, para recuperarlo, se debe utilizar `JSON.parse()`.

○ Verdadero ○ Falso

**Verdadero**

Convertir un array (o cualquier objeto complejo) en una cadena de texto utilizando `JSON.stringify()` es esencial para almacenarlo en localStorage, ya que localStorage solo almacena datos en formato de texto. Luego, al recuperar el array, se utiliza `JSON.parse()` para convertir esa cadena de texto de nuevo en un array. Este proceso de serialización y deserialización asegura que las estructuras de datos complejas se mantengan intactas y utilizables después de ser almacenadas.